

Python Code	Explanation
Load & Clean NRI Data	
<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn as sns import geopandas as gpd</pre>	This code uploads the necessary packages into Python for this project as abbreviations. Pandas for the analyzation of data, NumPy for numerical computing, matplotlib.pyplot for plots, seaborn for statistical graphing, and geopandas to read geospatial data.
<pre>print("===== ====") print("1. LOADING AND CLEANING THE NRI DATA") print("===== ====")</pre>	These lines display the text within the quotation marks to the user.
<pre>#Load the NRI datasets (Now correctly named state_tracts...) print("Loading NRI files...") nri_ca = pd.read_csv('state_tracts_California.csv') nri_mo = pd.read_csv('state_tracts_Missouri.csv')</pre>	The first line is a note discussing the renaming of state tracts. The second line reads the state tract csv for California and names the variable nri_ca. The third line follows the same format but for a different state, Missouri.
<pre># Combine into one NRI dataset nri_df = pd.concat([nri_ca, nri_mo], ignore_index=True)</pre>	This line of code combines the California and Missouri csvs into one called nri_df. In this same line the existing index labels are removed and replaced by a new default.
<pre># Smart FIPS column handling for NRI (It can sometimes be FIPS or TRACTFIPS) nri_fips_col = 'TRACTFIPS' if 'TRACTFIPS' in nri_df.columns else 'FIPS'</pre>	The first line is a note about NRI column handling. The second line looks for a data frame with a column named TRACTFIPS, if its there it uses it, otherwise it uses FIPS.
<pre>nri_df['JOIN_FIPS'] = nri_df[nri_fips_col].astype(str).str.zfill(11)</pre>	This line continues with column handling. It uses the column that code above chose and converts its numeric values into strings with a length of 11.
<pre># Handle Missing Values in NRI (Filling numeric NAs with the median)</pre>	The first line is a note. The next line looks for all numeric values in the columns within

<pre> numeric_cols = nri_df.select_dtypes(include=[np.number]).columns nri_df[numeric_cols] = nri_df[numeric_cols].fillna(nri_df[numeric_cols].median ()) </pre>	the NRI data frame. The final line replaces all NaNs values with the median of each column.
Load & Standardize NRI Data with SVI Data	
<pre> print("Loading NRI files...") nri_ca = pd.read_csv('state_tracts_California.csv') nri_mo = pd.read_csv('state_tracts_Missouri.csv') nri_df = pd.concat([nri_ca, nri_mo], ignore_index=True).copy() </pre>	The first line displays the message in quotations. The next two lines reload the state csvs as they were. The nri_df is also reloaded but as an independent data frame from the original.
<pre> # Rename the NRI state column to our universal name # Using 'STATEABBRV' as you identified nri_df = nri_df.rename(columns={'STATEABBRV': 'STATE_ABBR'}) </pre>	The two notes clarify what is being renamed. The last line of code looks for STATEABBRV in the columns and renames it to STAT_ABBR. This rename can be found in the data frame nri_df.
<pre> #Create the 11-digit FIPS for joining nri_df['JOIN_FIPS'] = nri_df['TRACTFIPS'].astype(str).str.zfill(11) </pre>	The note explains what is needed for joining digits. The second line converts every value in the TRACTFIPS column and converts to a string. The string will fill to 11 characters and store the result into a new column called JOIN_FIPS
<pre> # 2. LOADING AND STANDARDIZING SVI DATA print("Loading SVI files...") svi_ca = pd.read_csv('California.csv') svi_mo = pd.read_csv('Missouri.csv') </pre>	The first line is a note about code loading and standardizing SVI data. The second line reads the California csv and names the variable svi_ca. The third line follows the same format but for a different state, Missouri.
<pre> svi_df = pd.concat([svi_ca, svi_mo], ignore_index=True).copy() </pre>	This line of code combines the state csvs into one called svi_df. This is saved as an

	independent copy from the original.
#Rename the SVI state column to our universal name svi_df = svi_df.rename(columns={'ST_ABBR': 'STATE_ABBR'})	The note discusses rename again. The last line of code looks for ST_ABBR in the columns and renames it to STATE_ABBR. This rename can be found in the data frame svi_df.
#Create the 11-digit FIPS for joining (usually 'FIPS' in SVI) svi_df['JOIN_FIPS'] = svi_df['FIPS'].astype(str).str.zfill(11)	A note about FIP digit joining. The second line converts every value in the FIPS column and converts to a string. The string will fill to 11 characters and store the result into a new column called JOIN_FIPS within the svi_df frame
# 3. MERGING #We only need a few columns from SVI to avoid clutter svi_subset = svi_df[['JOIN_FIPS', 'RPL_THEMES']] combined_df = pd.merge(nri_df, svi_subset, on='JOIN_FIPS', how='inner')	The first two lines are notes. The svi_subset selects the JOIN_FIPS and RPL_THEMES. The combined_df merges the first line of code with nri_df to create a new data frame.
Calculate Alternative Risk Score	
print("===== ====") print("3. CALCULATING THE ALTERNATIVE RISK SCORE") print("===== ====")	This chunk of code prints out the text/symbols within the quotation marks. This is what the user will see.
# We calculate Alt Risk = EAL_VALT (Total Expected Annual Loss) * SVI Percentile (RPL_THEMES) # Note: if EAL_VALT isn't found, replace it with the specific Loss # column you want to use loss_col = 'EAL_VALT' if 'EAL_VALT' in combined_df.columns else 'EAL_SCORE' combined_df['ALT_RISK_RAW'] = combined_df[loss_col] * combined_df['RPL_THEMES']	The hashtag lines are notes about the following calculations. The next three lines of code pick the correct loss column (EAL_VALT or EAL_SCORE) and multiply it by RPL_THEMES, storing the result in ALT_RISK_RAW.
# Convert raw alternative risk into a percentile ranking (0 to 100) # to directly compare with NRI combined_df['ALT_RISK_SCORE'] = combined_df['ALT_RISK_RAW'].rank(pct=True) * 100	The first two are notes about conversion. The line of code converts the raw alternative risk values into percentile scores (0–100) by ranking ALT_RISK_RAW within the

	dataset. The values are then multiplied by the percentile rank by 100, storing the result in ALT_RISK_SCORE.
IDM Simulation	
# Calculate bias: Positive means NRI overestimates relative to # our method, Negative means NRI underestimates combined_df['RISK_DIFFERENCE'] = combined_df['RISK_SCORE'] - combined_df['ALT_RISK_SCORE']	The first two lines are notes about the calculations where positive values mean NRI overestimates, and negative values mean NRI underestimates. The second line computes the difference between the NRI risk score and your alternative risk score and store it in RISK_DIFFERENCE.
# Table 1: Average Risk Scores by State # We use 'STATE_ABBR' now state_summary = combined_df.groupby('STATE_ABBR')[['RISK_SCORE', 'ALT_RISK_SCORE', 'RISK_DIFFERENCE']].mean().round(2)	The first two lines are notes about average risk scores. The third line of code calculates the mean RISK_SCORE, ALT_RISK_SCORE, and RISK_DIFFERENCE for each state by grouping the data on STATE_ABBR, rounding the results to two decimals. The output is stored in the state_summary.
print("\n--- Summary Table 1: Average Risk Scores by State ---") print(state_summary)	The first line prints out the text within the quotation marks. The second displays a visual of the state summary table.
# Table 2: Top 10 Counties with Highest Scoring Divergence # NRI usually uses 'COUNTY' for the county name county_summary = combined_df.groupby(['STATE_ABBR', 'COUNTY'])[['RISK_SCORE', 'ALT_RISK_SCORE', 'RISK_DIFFERENCE']].mean().round(2)	The first lines are notes about scores of the top 10 countries. The next line of code groups the data by state and county. Next it computes the mean RISK_SCORE, ALT_RISK_SCORE, and RISK_DIFFERENCE for each

	group, rounds the results to two decimals, and stores the output in county_summary.
<pre> bias_table = county_summary.sort_values(by='RISK_DIFFERENCE', ascending=False).head(10) print("\n--- Summary Table 2: Top 10 Counties with Highest Scoring Divergence ---") print(bias_table) </pre>	This chunk of code sorts the county summary by RISK_DIFFERENCE in descending order, take the top 10 counties with the largest divergence, and print the resulting table.
Visualize The Data	
<pre> print("===== ===") print("5. VISUALIZING THE DATA (FIGURES)") print("===== ===") </pre>	These lines print out the text/symbols within the quotation marks.
<pre> plt.figure(figsize=(10, 6)) # Create the bias column if you haven't: combined_df['BIAS'] = combined_df['RISK_SCORE'] - combined_df['ALT_RISK_SCORE'] sns.violinplot(data=combined_df, x='STATE_ABBR', y='RISK_DIFFERENCE', palette="Set2") </pre>	Here the violin plot is created and is showing the distribution of RISK_DIFFERENCE for each state by plotting STATE_ABBR on the x-axis and RISK_DIFFERENCE on the y-axis using Seaborn, after initializing the figure size.
<pre> plt.axhline(0, color='black', linestyle='--') # The "No Bias" line plt.title('Distribution of Model Bias: NRI vs. Alternative Definition') plt.ylabel('Score Difference (Bias)') plt.savefig('Bias_Distribution_Violin.png') print("Saved scatter plot as 'Risk_Comparison_Scatter.png") </pre>	These lines add a horizontal reference line at zero, set the plot title and y-axis label, save the violin plot as Bias_Distribution_Violin.png, and print a confirmation message.
Create The Sensitivity Analysis Map	
<pre> print("===== ===") print("6. GENERATING THE SENSITIVITY ANALYSIS MAP") </pre>	These lines of code print out the text/symbols within the quotation marks.

<pre>print("===== ==")</pre>	
<pre>try: # 1. Load the shapefile gdf = gpd.read_file('NRI_Shapefile_CensusTracts.shp')</pre>	This code loads the tract-level shapefile into a GeoDataFrame using <code>gpd.read_file()</code> inside a try block.
<pre># 2. Filter for CA and MO and standardize the FIPS key gdf = gdf[gdf['STATEABBRV'].isin(['CA', 'MO'])].copy() gdf['JOIN_FIPS'] = gdf['TRACTFIPS'].astype(str).str.zfill(11)</pre>	The first line is a note about filtering. The next line filters the shapefile to include only California and Missouri, then create a standardized 11-digit FIPS join key by zero-padding the TRACTFIPS column and storing it in JOIN_FIPS.
<pre># 3. CRITICAL FIX: Drop the original NRI data columns from the shapefile # so they don't clash with our analysis data columns. # We keep ONLY the geometry and the JOIN_FIPS. gdf = gdf[['geometry', 'JOIN_FIPS']]</pre>	Here the code keeps only the geometry and JOIN_FIPS columns in the shapefile GeoDataFrame so none of the original NRI attribute fields conflict with your cleaned analysis dataset.
<pre># 4. Merge with your combined_df # This ensures RISK_SCORE, ALT_RISK_SCORE, and RISK_DIFFERENCE come from your CSV analysis geo_df = gdf.merge(combined_df, on='JOIN_FIPS', how='inner') if geo_df.empty: print("Error: The merge resulted in an empty dataframe. Check if JOIN_FIPS values match.") else: print(f"Success! Mapping {len(geo_df)} tracts.")</pre>	In this chunk the code merges the cleaned shapefile with your analysis DataFrame on JOIN_FIPS using an inner join, then check whether the merge produced any rows and print either an error message or a success message with the tract count.
<pre># 5. Create Side-by-Side Plots fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(22, 10))</pre>	Step five initializes a figure with two side-by-side subplot axes (ax1 and ax2) arranged in one row and two columns,

	using a total figure size of 22×10 inches.
<pre># Map 1: Original NRI Risk # Note: If 'RISK_SCORE' still fails, use 'RISK_SPCTL' which is the percentile version nri_col = 'RISK_SCORE' if 'RISK_SCORE' in geo_df.columns else 'RISK_SPCTL'</pre>	In these lines we select the correct NRI risk column for mapping by using RISK_SCORE if it exists; otherwise fall back to the percentile version RISK_SPCTL, and store the chosen column name in nri_col.
<pre>geo_df.plot(column=nri_col, cmap='YlOrRd', legend=True, ax=ax1, legend_kwds={'label': "Original NRI Risk Score", 'orientation': "horizontal"}) ax1.set_title(f"Standard FEMA NRI Risk ({nri_col})", fontsize=15) ax1.set_axis_off()</pre>	In this line of code, we are plotting the selected NRI risk column on the first map using a yellow-to-red color scale, adding a horizontal legend, setting the title, and hiding the axis for a clean map display.
<pre># Map 2: Your Calculated Bias geo_df.plot(column='RISK_DIFFERENCE', cmap='coolwarm', legend=True, ax=ax2, legend_kwds={'label': "Bias (NRI Score - Alt Score)", 'orientation': "horizontal"}) ax2.set_title("Categorical Bias: Where NRI differs from Proposed Model", fontsize=15) ax2.set_axis_off()</pre>	This code is to plot the second map by coloring tracts according to RISK_DIFFERENCE using a cool-to-warm diverging colormap. We add a horizontal legend, set the title, and hide the axis to see the NRI over- or under-estimates risk relative to your model.
<pre>plt.tight_layout() plt.savefig('Risk_Sensitivity_Comparison.png', dpi=300) plt.show() print("Comparison map saved successfully.")</pre>	This tightens up the layout, saves the side-by-side comparison figure as Risk_Sensitivity_Comparison.png at 300 dpi. This is then displayed and prints a confirmation message

	indicating the comparison map was saved successfully.
except Exception as e: print(f"Mapping failed again. Error: {e}")	Lastly, this catches any exception raised during the mapping workflow and prints a clear error message containing the exception text, so you can diagnose what failed.